

Rapport Projet Innovant 2025



Université Claude Bernard



Alexandre Cottier · Anthony Venet · Émilie Vernay · Pierre Manoel · Théo Labrosse

Lien du projet sur Git Forge Lyon 1

0 - Répartitions des tâches

Alexandre Cottier :

- Développement des scripts de détection et d'IA des robots
 - Vision par ordinateur (segmentation couleur, ArUco, homographies, tracking CSRT)
 - IA (Sélection de cible, pilotage par différences angulaires, machine à états)
 - Outils de débogage visuel (Overlay vidéo, plan 2D)

Anthony Venet :

- Conception originale du robot "Voleur"
- Développement technique et construction manuelle partielle du robot "Voleur"
 - Fabrication de la structure non-finale, des moteurs et du bras pour le prototype

Émilie Vernay :

- Mise en place du Raspberry Pi avec le WiFi
- Script de création des clefs SSH pour le Pi et les robots
- Système de déploiement automatique du code sur le Pi et les robots

Pierre Manoel :

- Conception originale du robot "Chasseur"
- Développement technique et construction manuelle intégrale du robot "Chasseur"
 - Fabrication de la structure, des moteurs, de la pince et des capteurs
- Développement technique et construction manuelle majoritaire du robot "Voleur"
 - Fabrication de la structure, des moteurs et des capteurs. Aide sur le bras

Théo Labrosse :

- Mise en place du serveur/API web en REST
- Mise en place de la télécommande (fetch depuis l'API)
- Mise en place de la communication par websocket entre les robots et la télécommande/la logique IA
- Développement du programme partagé des robots
 - Cela inclut le code de connection (serveur websocket)
 - Le code contrôlant les moteurs des robots
 - Le code contrôlant les actions automatiques des robots (bras qui tourne, pince qui se ferme)
 - Calibration des moteurs (niveau d'ouverture de la pince)
 - Autres éléments : musique, contrôle du canon

I - Vision par ordinateur

Ce qui a été utilisé pour la démo :

Premièrement, nous avons commencé par le plus simple, c'est-à-dire **détecter les balles** de différentes couleurs (rouge et bleu) et de bonne taille (ni trop petite, ni trop grosse) :

- On convertit les couleurs au format HSV, puis on applique un seuillage double (rouge : deux intervalles, bleu : un intervalle).
- On nettoie le résultat par opening morphologique (`cv2.MORPH_OPEN`).
- On extrait les contours, puis on filtre par aire, rayon et circularité $\geq 0,70$ pour ne garder que les formes quasi-circulaires.
- Sur la fenêtre : affichage du cercle autour de chaque boule détectée + texte $R_r C_c$ (r = rayon, c = circularité), ce qui nous a aidé pour bien définir la plage de configuration. L'idée étant de limiter le plus possible ce qu'on détecte (on ne veut pas d'objets indésirés qui soient trop ronds ou rouges/bleus).

Pour la **détection des robots**, nous avons opté pour des codes ArUco, ce qui permettait d'utiliser une simple librairie pour la détection du robot malgré la faible résolution des caméras. Selon l'angle et la distance, les codes étaient peu visibles et avaient du mal à être détectés en continu :

- On utilise `cv2.aruco.detectMarkers` avec un dictionnaire `DICT_6X6_250` et des paramètres sévères (raffinement sous-pixel, contraintes de périmètre) pour essayer d'ajuster au mieux la détection face aux contraintes de qualité du flux vidéo.
- Les IDs 20 et 30 servent d'identifiants de robots même si nous avons les IDs 1 et 2 initialement. Vu que d'autres groupes avaient des ID similaires, nous avons généré de nouveaux ID pour être sûrs que la détection avec plusieurs robots se passe bien.
- Calcul du centre (Barycentre des 4 coins) + orientation : vecteur (Centre $\rightarrow$ Coin 0), converti en degrés (Offset $+40^\circ$ pour aligner avec l'avant réel du robot), ce qui nous est utile par la suite pour l'IA des robots.

Pour le **suivi des balles**, vu que l'on définit une normale à notre robot lors de sa détection, on peut donc tracer une flèche entre le centre du robot et la balle la plus proche. Il va se mettre à jour selon s'il trouve une balle plus proche entre-temps, mais nous avons également rajouté quelques détails en termes de détection des balles :

- Petite data-association maison : on cherche la détection la plus proche (< 20 px) d'une trace existante. Ladite "trace" est une balle qui a été présente à cet endroit à un moment récent. On tolère une courte latence pour éviter les fractures si la détection flashe ou qu'il perd la balle momentanément. Ainsi, si la balle est perdue de vue pendant une demi-seconde, le robot maintient son cap. On met ensuite à jour une moyenne glissante et on incrémente un compteur de persistance.
 - Une détection doit tenir ≥ 3 frames d'affilée pour être acceptée (on ne veut pas considérer des fragments ou des petites erreurs de notre détection, pour des balles à tracker) ; elle disparaît après 20 frames sans match (dans quel cas on retourne sur la vraie position de la balle).

Avec cette même logique, on souhaitait recourir au même principe pour le suivi des robots. Néanmoins, selon la distance, nos robots auraient eu plus ou moins de mal à être détectés, ou surtout à maintenir cette détection (sans flash ou perte de reconnaissance du robot, ce qui rendait très dure l'utilisation de l'IA pour les balles). On a donc opté pour une **détection de nos robots via marqueur**. Puis, notre programme utilise un tracker pour garder la position du robot et son orientation. On conserve cette nouvelle norme pour l'IA, et ainsi le tracker se met aussi à jour selon la position du robot (et de son orientation bien entendu). Si jamais notre tracker perd le robot, on relance la détection avec le code ArUco et dès qu'il le retrouve, il recommence sa détection via le tracker :

- Création dynamique d'un tracker CSRT (`cv2.TrackerCSRT_create`).
- On initialise la bounding-box autour de la dernière position connue.
- Problème : la détection via tracker n'est pas 100% fiable comme notre détection via code ArUco (si la qualité aurait été meilleure), il peut légèrement se tromper sur l'orientation du robot selon l'angle. Il est donc

assez difficile d'utiliser cette fonction en permanence quand on requiert absolument des données fiables sur l'orientation actuelle du robot.

On a créé un **masque noir/blanc** limitant la détection au contenu de l'arène. On a aussi créé un **plan 2D** auquel, via un autre script, on a défini les 20 coins de l'arène, pour pouvoir retranscrire la géométrie dans le plan 2D. Cela nous permet de combiner les affichages des deux caméras (d'angles différents) en un unique plan de l'arène, dont le contenu (balles/robots et leurs positions) est calculé le plus précisément possible. Il s'agit de l'homographie du terrain : on transforme des coordonnées pixel en coordonnées « plan » (composant une vue de dessus fictive du terrain). Cependant, les balles sur le terrain se dédoublaient en premier temps car les caméras d'angles différents étaient en désaccord sur les positions des balles. Pour y remédier, on a recouru aux ajustements suivants :

- Calcul de deux homographies : H_{top} pour la moitié haute et H_{bot} pour la moitié basse de l'image.
- Les points de références proviennent d'un gabarit circulaire (monde) et des sommets détectés sur la table.
- `interpolate_point` réalise un blend linéaire entre H_{top} et H_{bot} , selon la position verticale du point, ce qui a pour effet de réduire l'erreur de perspective sur les balles.

Et enfin, avant de passer à l'IA, on veut connaître la **direction du robot**, comme dit précédemment. On fixe sa norme par rapport à son code/le tracker et avec ceci, on va pouvoir savoir comment diriger le robot selon sa cible :

- Calcul de l'angle (Centre robot $\rightarrow$ Balle), puis différence d'angle livrée au module décisionnel (dans la suite avec l'IA).
- Dessin du vecteur de direction sur le flux pour validation visuelle (à des fins de maintenance et débogage du robot).

Ce qui a été développé mais pas utilisé pour la démo :

Nous avons opté en cours de route pour un **marqueur par couleur**, qui était beaucoup mieux détecté par les caméras car moins besoin de résolution pour le voir. Nous étions partis sur un trait épais d'une première couleur (autre que rouge et bleu pour éviter les confusions) et une barre horizontale d'une seconde couleur qui désignait la tête du robot dans notre programme. Les résultats semblèrent concluant, mais nous avons préféré nous en tenir aux codes ArUco car les codes couleur étaient susceptibles de causer des bugs (ex : si l'une des couleurs du code est mal détectée).

Par rapport au **suivi des balles**, nous avons fait en sorte de pouvoir tracker une balle, c'est-à-dire que le robot suive une balle et la poursuive jusqu'à l'avoir attrapée (donc pas de mise à jour selon s'il a trouvé une autre balle plus proche) car selon l'angle de caméra, il pouvait arriver que le robot détecte 2 balles équidistantes et qu'il ne sache pas laquelle prendre, causant une boucle infinie.

A la base, nous devions nous servir du plan 2D pour la direction, etc., sauf qu'à cause des imprécisions des angles, cela devenait trop hasardeux, voire impraticable avec le tracking du robot. Nous avons donc opté pour l'analyse sur le flux directement plutôt que sur le plan 2D fraîchement créé. Cela dit, avec un peu plus de temps, nous aurions pu utiliser le plan 2D.

II - Réflexion / IA des robots

États et prise de décision comportementale :

L'IA des robots repose sur un modèle d'états :

- **SEARCHING** : Détection et suivi de la balle cible, ajustement d'angle et déplacement.
 - ▶ Ils envoient donc des messages à la websocket du type "Forward", "Stop",... pour ce qui est du déplacement, et "Open", "Shoot",... pour ce qui est des fonctionnalités supplémentaires.
 - ▶ Concrètement, les robots envoient des commandes seulement quand le programme leur dit qu'il **faut** exécuter une nouvelle commande (pas de spam de la websocket).
 - ▶ Selon leur normale, ils envoient "turnleft" ou "turnright" pour s'ajuster à la position de la balle, puis "forward" pour avancer vers la balle.

- Ensuite, selon le robot :
 - **Le Chasseur** rentre dans la balle et ferme sa pince (gérée côté websocket avec double capteur), puis il envoie un message “CONFIRMED” au programme d’IA pour indiquer qu’il tient une balle dans sa pince, ce qui le fait transitionner vers la phase “DELIVERING”. Le message “CONFIRMED” est envoyé uniquement quand le capteur de pression dans la pince affirme qu’une balle a été attrapée. Si la pince s’est fermée mais qu’aucune balle n’a été confirmée, celle-ci s’ouvre à nouveau d’elle-même.
 - **Le Voleur**, lui, agit de la même manière mais n’a pas besoin de basculer dans un autre état dès la première balle. Il possède un compartiment pouvant accueillir jusqu’à 5-6 balles; lorsque le compartiment est plein (c’est-à-dire qu’un des capteurs est poussé au fond) alors il envoie lui aussi un message “CONFIRMED” et entre en phase “DELIVERING”.
- **DELIVERING** : Retour au point de dépôt, exécution d’une séquence (reculer, ouvrir pince -> que l’on a mise en place pour la démo, sinon il suffirait de fixer un point et il s’y déplacerait et exécuterait une série d’actions pour le delivering). Après avoir relâché la/les balles, les robots repassent dans l’état “SEARCHING”.

Nous avons aussi ajouté un canon à billes sur le **Chasseur**. Lors de la démo, le canon tirait à chaque fois qu’il attrapait une balle.

III - Interfaces

Dès le début du projet, nous voulions mettre en place une interface web pour pouvoir tester le robot et communiquer avec. À l’origine du projet, cette interface prenait la place d’une API à travers laquelle le robot pouvait envoyer des logs et les valeurs de ses capteurs avant de fetch les ordres envoyés par le programme de réflexion qui postait les ordres des robots sur l’API. Beaucoup de temps de réflexion a été consacré à cette API, avant qu’elle ne puisse finalement être testée sur les robots, et que les défauts ne soient mis en évidence. Entre l’envoi d’un ordre et l’action du robot, plus d’1 seconde s’écoulait. Résultat : même avec une télécommande, il était impossible de contrôler n’importe lequel de nos robots de sorte à lui faire attraper une balle.

Après cette réalisation, nous avons changé de méthode, et décidé d’utiliser les websockets de Python pour toutes les communications avec le robot. Cette section du rapport va détailler le système actuel mis en place pour communiquer avec les robots.

Serveur Websocket

Mise en place

Le serveur websocket est lancé depuis le fichier `connector.py` du code des robots. Il met en place une websocket sur les 2 robots, et gère les connexions et déconnexions de ces websockets.

C’est depuis cette classe de connexion que les ordres vont être reçus et envoyés à la classe gérant la logique moteur du robot spécifique.

Communication bi-directionnelle

La communication sur ces websockets est bi-directionnelle, dans un sens les robots reçoivent les ordres pour se déplacer. Tant qu’un ordre n’est pas changé ou remplacé par un autre, le robot qui reçoit l’ordre va continuer à l’exécuter. Par exemple, envoyer une seule fois l’ordre **Forward** va faire avancer le robot en ligne droite jusqu’à ce qu’il reçoive un ordre indiquant le changement d’état, comme **Stop** ou **Left**, faisant respectivement s’arrêter et tourner le robot à gauche. Cela a été mis en place pour éviter 2 problèmes :

- Avoir besoin de spammer les ordres pour que le robot comprenne (surtout utile pour la télécommande).
- Les saccades dans les mouvements, se produisant en particulier lorsque le robot doit effectuer un déplacement ou une rotation de paramètre (distance/angle) très précis. Cette méthode permet d’avoir un mouvement fluide. Si le robot devait faire preuve de grande précision, alors stocker les ordres dans une liste d’exécution FIFO aurait pu être nécessaire. Mais comme nous utilisons des caméras peu précises et qu’il y avait de grandes chances que les balles soient en mouvement au moment de les attraper, nous avons décidé

de partir sur un système moins précis, mais plus fluide et qui aurait du coup plus de chance de pouvoir suivre les balles. Nous verrons dans la prochaine section que les robots ont été construits dans ce but.

Dans l'autre sens, les robots envoient des informations sur leurs capteurs. Une logique a été mise en place dans les robots pour déterminer s'ils avaient attrapé une balle / étaient pleins. C'est cette information qui est envoyée sur la websocket afin de permettre à la partie logique du projet de décider de la prochaine marche à suivre :

- "NOTHING" quand le robot n'a pas de balle (Chasseur) / n'est pas plein (Voleur).
- "CONFIRMED" quand la logique interne du robot confirme qu'une balle a été attrapée (Chasseur) / que le robot est plein (Voleur).
- "SEEN" est un message spécifique au robot Chasseur doté d'un capteur ultrasonore. Quand il voit la balle, ou tout autre objet suffisamment proche, il va l'indiquer sur la websocket. Ce message est désormais seulement à titre de débogage et est traité comme "NOTHING" par la partie IA du projet.

Qui s'en sert ?

Deux outils se servent des websockets dans ce projet.

- La télécommande qui peut être accéder depuis l'api Crowserver à <http://localhost:8080/client/remoteWS> quand l'API web est lancée. Il s'agit d'une télécommande qui permet de contrôler les 2 robots à la fois. Elle se connecte aux websockets des 2 robots. Elle va aussi afficher les informations de niveau de capture ("NOTHING", "SEEN", "CONFIRMED") de la balle pour permettre de déboguer.
- La partie IA du projet se connecte aux robots pour leur envoyer des ordres, ce en passant par les websockets.

Travail n'étant plus utilisé

Les pages de debug

Quand nous pensions utiliser l'API web, nous avons construit une série de pages pour déboguer le programme et observer l'état des robots. L'API a été construite avec un simple système de base de données. A partir d'un singleton et de DAOs, elle permettait d'enregistrer les logs et messages d'erreur des robots, mais aussi les ordres qu'ils recevaient et si ceux-ci étaient traités. Il y avait aussi une page séparée qui permettait de voir les valeurs des capteurs.

Tout ce système a été abandonné au profit des websockets, mis à part la page de la télécommande qui avait été construite en même temps. Cette page a simplement reçu une mise à jour avec un code JS plus complexe lui permettant de se connecter par websocket aux robots.

Remarques

Mettre en place ces serveurs de websockets sur les robots n'aurait pas été si simple si nous n'avions pas, dès le début, mis en place le WiFi et fixé les adresses IP des robots.

IV - Les 2 robots

Le Chasseur

Prototype de robot original conçu par Pierre, le "Chasseur" a été spécialement pensé pour répondre aux objectifs principaux du projet : attraper une balle et l'emmener dans son camp. Profitant d'une large liberté de conception, nous avons opté pour une structure solide, capable de résister aux assauts de nos concurrents tout en conservant une bonne mobilité.

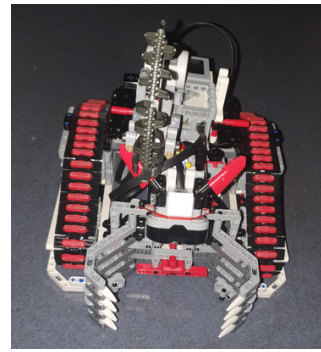
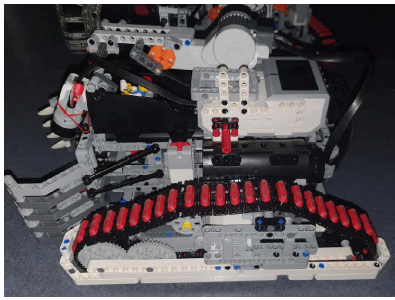


Figure 1: Image du “Chasseur”

Moteurs :

A la base, nous avons commencé par la création du moyen de déplacement. Nous avons opté pour deux moteurs qui actionnent chacun une chenille, permettant au robot de se déplacer facilement. Le choix des chenilles (à la manière d'un tank) facilite la programmation des mouvements : pour aller tout droit, les deux moteurs tournent en avant ; pour reculer, ils tournent en arrière ; pour tourner, un moteur avance pendant que l'autre recule. Simple et efficace.

Afin de gagner de la place, nous avons intégré les moteurs directement dans les chenilles, ce qui a légèrement complexifié le développement, mais offert plus de liberté pour le reste de la structure. À noter que dans le design final, la vitesse du robot est directement liée à celle des moteurs. Un ancien prototype utilisait un système d'engrenage qui, bien qu'il augmentait le couple des moteurs et donc la vitesse, réduisait la force et entraînait de fréquentes défaillances mécaniques, notamment lorsque les robots tournaient sur eux même et changeaient de direction rapidement. Ce système a donc été abandonné.

Pince :

Élément central du robot : sa pince. Nous avons choisi de rendre la pince large et solide, afin de faciliter la capture des balles (et parfois de robots adverses... mais c'est un détail). À l'intérieur de la pince, un capteur de pression permet de détecter la prise d'une balle et d'éviter que le moteur ne force inutilement.

Après de nombreux ajustements, nous avons également ajouté un capteur ultrasons au-dessus de la pince. Celui-ci déclenche la fermeture automatique dès qu'un objet pénètre dans la zone. Ce système a toutefois un inconvénient : il peut également attraper d'autres robots (ou des mains) passant trop près.

Un système dans le code du robot permet de réouvrir automatiquement la pince si le capteur de pression n'est pas activé, car cela indiquerait que la pince s'est fermée mais n'a pas attrapé la balle. Cela permet d'éviter les faux positifs et les cas où la balle échappe à la pince au moment où elle se ferme. Dans des cas précis, des faux négatifs peuvent avoir lieu, quand la balle n'est pas suffisamment pressée contre le capteur.

Structure :

Enfin, pour relier tous ces éléments, nous avons conçu une structure robuste et relativement lourde, avec le cœur EV3 positionné entre les moteurs et derrière la pince. En prévision des attaques potentielles, des protections ont été ajoutées, notamment autour des chenilles pour éviter d'être escaladé ou renversé, ainsi qu'une dissimulation partielle du câblage pour éviter tout débranchement.

Enfin, afin de faciliter la programmation et la maintenance, nous avons intégré une certaine modularité à la structure. Le cœur et son câblage peuvent ainsi être retirés et remontés facilement, sans risque de casse.

Bonus :

Bien qu'inutiles d'un point de vue fonctionnel, nous nous sommes permis d'ajouter quelques détails esthétiques et fun à notre robot pour le rendre plus vivant. Parmi eux : un pilote dans son cockpit, un minigun actionnable par bouton, ainsi que des cornes imposantes. Des touches de caractère qui n'apportent rien à la performance, mais tout au style.

Le Voleur

À l'opposé du "Chasseur", plutôt classique, le "Voleur" repose sur un concept radicalement différent, imaginé par Anthony. Fonctionnant comme une véritable prison à balles, son objectif est d'en bloquer un maximum à l'intérieur, soit pour les ramener plus tard à la base, soit pour simplement empêcher les adversaires de s'en emparer.

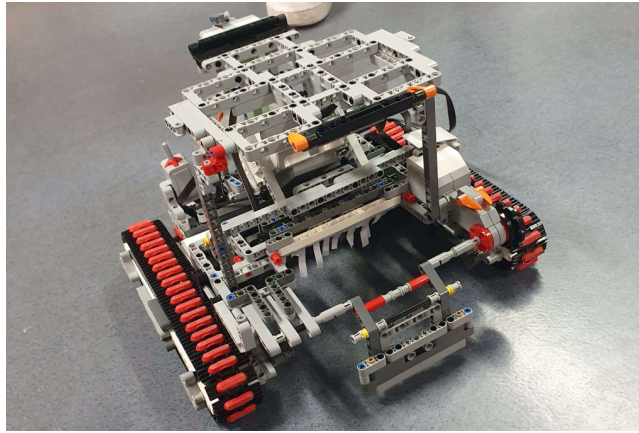


Figure 2: Image du "Voleur"

Moteurs :

Sur le plan technique, le Voleur se déplace exactement comme le Chasseur : les moteurs tournent en avant pour avancer, en arrière pour reculer, et dans des sens opposés pour tourner. La seule différence notable est que les moteurs ne sont pas placés dans les chenilles, mais sur les côtés intérieurs de celle-ci.

Structure :

Ce concept nécessitant un espace intérieur dégagé, son design est beaucoup plus simple, léger, voire fragile. Le principal défi de ce robot était de créer une suffisamment solide pour ne pas s'effondrer, tout en ayant suffisamment de place vide à l'intérieur pour stocker beaucoup de balles. Cela a nécessité une bonne répartition des masses, notamment via le placement stratégique du cœur EV3. Pour mieux soutenir le "plafond" de la prison à balles, un pied de support a été rajouté, doté d'une bille maintenue à son bout afin de ne pas freiner le robot dans ses déplacements.

En raison de la nature plus légère du robot, nous avons laissé les chenilles relativement à découvert, afin de ne pas complexifier inutilement la construction. Ont été ajoutées des protections minimales, pour empêcher les robots ennemis de se loger dans les chenilles du Voleur. Ce choix a aussi permis de réduire le poids global et de gagner en maniabilité.

Pince (ou bras) :

Contrairement à la pince du Chasseur qui attrape la balle au bon moment, le bras du Voleur tourne en continu de sorte à tirer les balles de l'avant vers l'intérieur, alors que le robot avance dans la direction de la balle. Les balles tirées par le bras sont ainsi poussées sous le robot qui leur sert de prison. Les révisions finales du Voleur ont limité le nombre de moteurs du bras utilisé de 2 à 1, suffisant pour effectuer la rotation continue.

Le Voleur peut "avalier" jusqu'à près de 6 balles avant d'atteindre sa capacité maximale. Lorsque cela se produit, les balles sont poussées par le bras, contre le fond de l'intérieur du robot, où s'y trouve une plaque de pression reliée à des capteurs. Quand ces capteurs sont enclenchés, le Voleur envoie le message "CONFIRMED", puis est en retour instruit de stopper la rotation du bras.

Le robot est ensuite destiné à soit :

- Avancer vers le camp, lever le bras, s'arrêter (faisant sortir l'essentiel des balles en utilisant la vitesse transmise par le robot), puis à tourner le bras dans le sens inverse pour faire sortir les balles.
- Plus simplement se "sacrifier" en allant dans le camp, y plaçant donc les balles par la même occasion. On assume alors que la quantité de balles récupérées est satisfaisante.

A noter : la détection du remplissage du robot peut ne pas toujours fonctionner, et le bras ne s'arrêtera alors pas d'essayer de tourner. Cela peut causer des régurgitations de balle indésirées, ou une gêne niveau moteur si le bras est bloqué par des balles.

Evolution du design :

L'idée initiale était de récupérer les balles à l'aide d'un bras "snap-fit" : le bras devait se baisser sur la balle en appuyant dessus, la faisant ensuite ressortir sur le dessus. (On imagine que la balle aurait pu être stoppée au bout du bras par une petite extension.) Une fois la balle sur le bras, celui-ci n'aurait eu qu'à bouger vers le haut pour faire rouler la balle en direction d'une cage située sur le dessus du robot. Le toit du robot aurait servi à cacher les balles. Cependant, plusieurs problèmes rendaient ce concept très peu praticable :

- Le robot aurait sans doute eu des problèmes d'équilibre (même avec des contrepoids, comme illustré sur l'image ci-dessous) avant même d'envisager que d'autres robots ne puissent le renverser.
- Le bras "snap-fit" aurait probablement envoyé la balle voler, par la nature du snap-fit. Il aurait alors été possible d'ajouter un "toit" au bras, mais cela aurait d'autant plus alourdi le bras, contribuant au déséquilibre.
- Cela est, si le bras n'aurait pas cédé ou interagi différent, puisqu'il s'agit de Lego (le bras aurait pu casser/se défaire, ou plus simplement ne pas être assez souple, etc.)

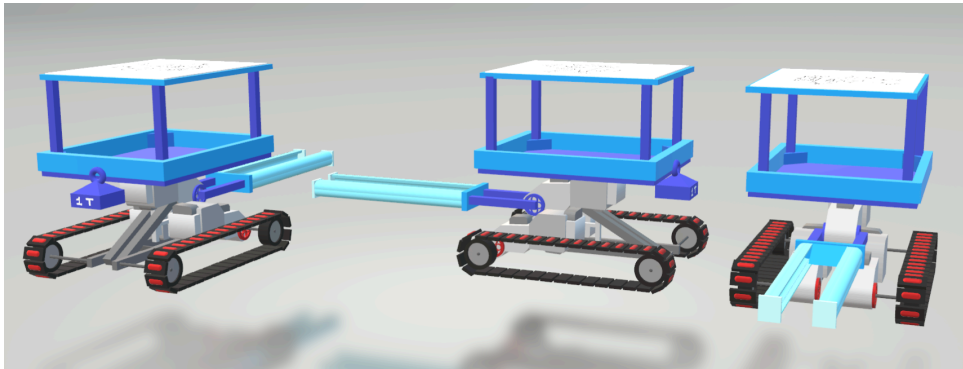


Figure 3: Premier design (non peaufiné) du "Voleur", modélisé dans Paint 3D.

Ensuite, basé sur ce design, vint l'idée de simplement cacher les balles sous le robot, évitant d'avoir à mettre des balles sur son dos (peu évident) et résolvant les problèmes mentionnés antérieurement. Cependant, le placement central du moteur pour un bras "avaleur" de balles fut maintenu pendant un certain temps. Fut ensuite décidé de placer un moteur de chaque côté du robot, principalement pour la symétrie, et d'attacher un bras rotatif entre ces deux moteurs. Peu avant la démonstration, l'un des moteurs a été retiré afin d'éviter des actions contradictoires entre les moteurs, ce qui les ferait forcer l'un sur l'autre et abîmerait également le bras rotatif qu'ils se partagent.

Bonus :

Un temps moins conséquent a été passé à la décoration du Voleur. Plutôt que de viser une esthétique élaborée, ce robot est doté de décorations minimales, incluant quelques pièces larges, ainsi qu'un dessin de "trollface" à l'arrière, attaché à une plaque de maintien en Lego similaire à celle utilisée pour les codes ArUco. La faible surcharge visuelle a aussi pour effet technique de ne pas alourdir le robot.

Anecdotiquement, une musique de près de 30 secondes se lance au démarrage du robot.

Le code

Le code des robots est écrit en Python. Les deux robots partagent un même code, mais chacun possède ses propres moteurs et logiques différentes. Pour prendre ceux-ci en compte, le fichier `config.json` permet au runtime de déterminer quel code doit être exécuté selon le robot.

Le code est séparé en 4 fichiers :

- `connector.py` où l'on retrouve la logique de connexion des websockets aux robots.
- `motor.py` où l'on retrouve le code des activations moteur.
- `test.py` qui permet d'effectuer des tests simples et de calibrer le moteur de la pince du Chasseur.

- `client.py` qui sert de script principal et permet de lancer en parallèle les threads des différentes parties de la base de code.

Connexion

Le programme de connexion consiste à ouvrir un serveur de websocket sur le port désigné et d'attendre la connexion d'un contrôleur (une télécommande ou l'IA). Deux fonctions vont tourner en parallèle dans cette classe :

- `receive_commands`
- `send_sensor_data`

Dont les noms sont plutôt explicatifs; la partie intéressante à noter est que c'est depuis ces 2 fonctions que les appels aux codes des moteurs sont effectués. Grâce à du polymorphisme, nous n'avons pas besoin de savoir à cette étape de quel robot il s'agit. Les deux ont les mêmes appels de fonctions et ne sont pas distingués dans ces fonctions.

Moteurs

Le fichier `motor.py` gère la logique interne des robots et gère les appels moteurs via la librairie **ev3dev2**. Pour cela, 3 classes sont mises en place :

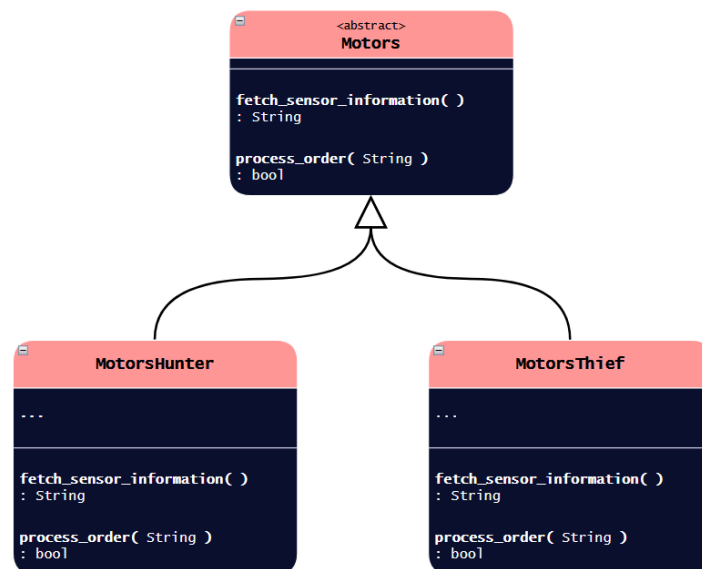


Figure 4: Diagramme UML des classes gérant les moteurs des robots

La classe `Motors` permet de mettre en place du polymorphisme, utile pour le code de connexion.

La fonction `process_order` lit une string correspondant à un ordre, l'ordre est lu par une série de `if/else` jusqu'à exécuter l'ordre en question. Ce système très simple à mettre en place offre deux avantages :

- **Extensibilité** : Il est très simple d'ajouter un nouvel ordre.
- **Contrôle** : Les moteurs sont indépendants les uns des autres, et changent d'état seulement quand un ordre entre en conflit avec un ordre précédent (par exemple : "Forward" contre "Stop"). Le résultat est que le dernier ordre est appliqué à tous les moteurs correspondants, ce qui évite d'avoir besoin d'annuler un ordre précédent. Cela permet aussi de séparer le contrôle de moteurs distincts, et par extension de contrôler les bras et pinces de manière distinctes des mouvements du robot, sans oublier la possibilité d'enchaîner les ordres sans aucun risque de conflits.

La fonction `fetch_sensor_information` est plus complexe et dépend fortement du robot. Elle lit les informations des capteurs rattachée aux robots, et établit une conclusion quant à l'état actuel du robot en fonction de ces valeurs. Pour rappel, les deux états utilisés par l'IA sont "CONFIRMED" et "NOTHING", indiquant respectivement le fait d'être rempli ou non de balles. Cette fonction est appelée à intervalles réguliers par `send_sensor_data` du code de connexion pour communiquer l'état du robot sur les websockets.

V - Infrastructure

Architecture du système

Notre système de contrôle des robots forme une boucle rétroactive. Les caméras récupèrent les données du monde. Ces données sont ensuite transmises à un serveur qui va analyser ces données pour y extraire la position des robots et la position des balles. Ce même serveur va envoyer des ordres de mouvement aux robots pour récupérer les balles. Ces mouvements sont capturés par les caméras et peuvent être corrigés en cas d'erreur de précision.

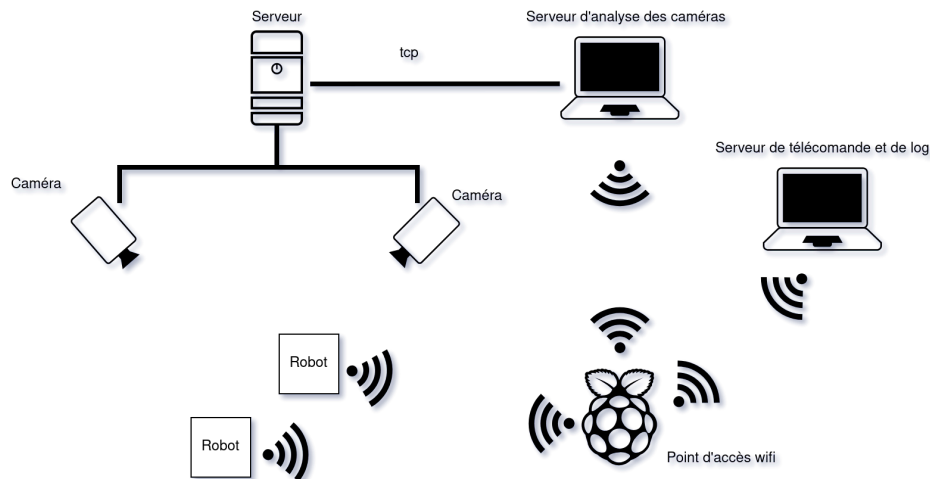


Figure 5: Schéma de l'infrastructure de communication Serveur ↔ Robot

Un des principaux défis techniques a été d'optimiser le temps entre l'acquisition des données et l'envoi d'ordres au robot afin de pouvoir corriger les erreurs d'imprécision rapidement.

- Une des premières améliorations a été de connecter le serveur d'analyse des caméras directement sur le réseau filaire sans le faire passer par la Raspberry Pi. Ici, nous utilisons la Raspberry Pi uniquement comme point d'accès WiFi. En effet, celui-ci n'est pas suffisamment puissant pour traiter les images en temps réel.
- La deuxième optimisation a été d'utiliser des websockets pour faire la communication entre le serveur et les robots. Les robots ouvrent chacun une websocket. Ensuite, le serveur d'IA et/ou le serveur de télécommande se connecte dessus. Ils peuvent alors envoyer des ordres aux robots, qui peuvent alors envoyer des messages de log sur leur websocket. Le temps de réponse via ce système est de l'ordre de la milliseconde, là où il était d'une seconde lors de notre tentative par requête HTTP.

Pour éviter de devoir re-récupérer l'IP des robots, nous leur avons assigné une IP statique sur le WiFi.

Outils de déploiement

Afin de faciliter la mise en place du système pour la configuration des robots et du Pi, nous avons créé plusieurs scripts.

- Création de clef SSH pour les équipements

Pour éviter de devoir entrer manuellement les mots de passe pour se connecter aux équipements, nous avons fait un script qui va créer des clefs SSH et les déployer sur les robots et le Pi. Une fois l'installation terminée, un message affiche les commandes pour se connecter à chaque équipement.

- Déploiement automatique du code

Grâce à la connexion via les clefs SSH, nous pouvons mettre en place un système de déploiement automatique. Pour cela, il faut placer un fichier de configuration dans chaque dossier qui doit être déployé. Ce fichier de configuration indique sur quel équipement déployer, quelle commande d'initialisation lancer, puis quelle commande d'exécution lancer.